

배포와 Render

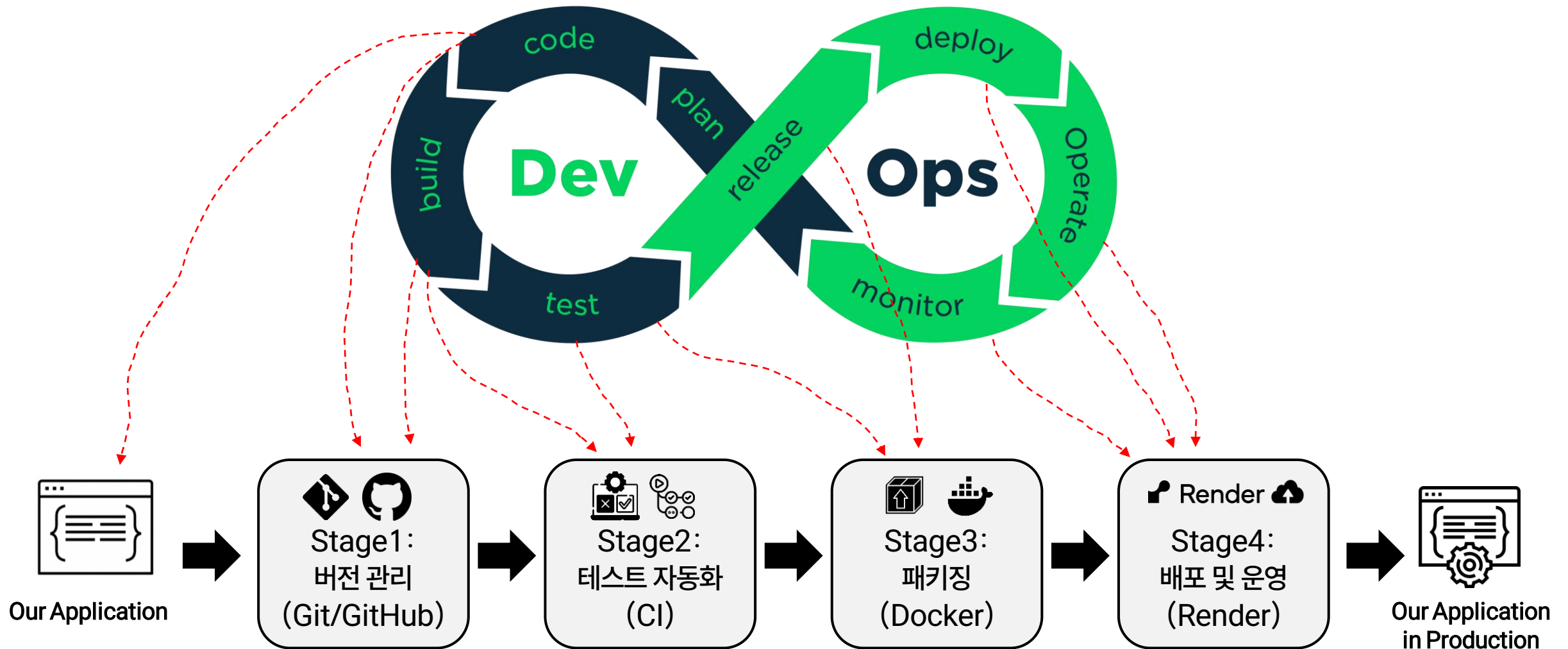
김미수

전남대학교 AI융합대학 인공지능학부

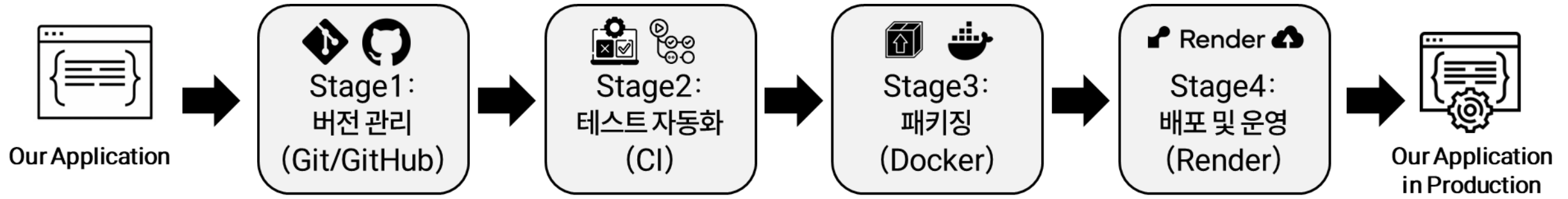
misoo.kim@jnu.ac.kr

<https://sites.google.com/view/semi-jnu>

수업에서 진행할 파이프라인

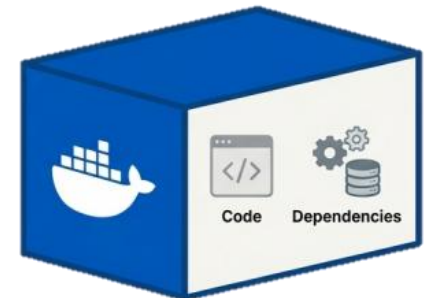


DevOps 파이프라인

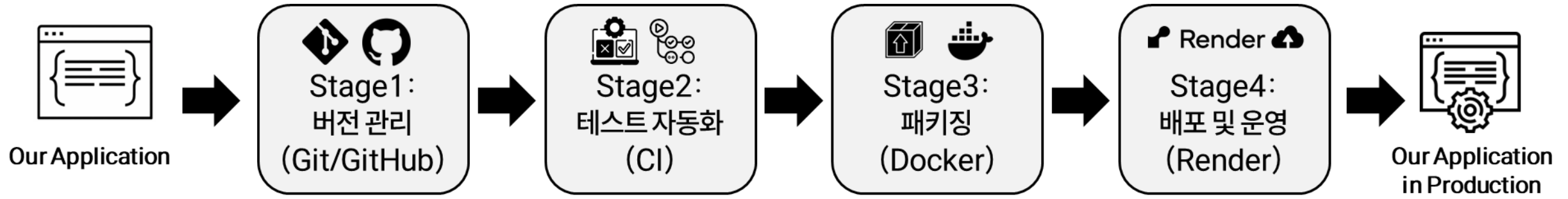


- 2단계: 일관성을 위한 패키징: “내 컴퓨터에서는 되는데...” 문제 해결

- 애플리케이션 + 실행 환경 (라이브러리, 종속성 등)을 하나의 ‘컨테이너’로 패키징
 - ▶ 이를 통해 개발, 테스트, 운영환경 어디서든 동일하게 실행되는 환경 구축
- 도구: GitHub Action 기반 Docker 이미지 빌드



DevOps 파이프라인



- **3단계: 배포와 운영 – 서비스하고 살아있는지/문제없는지 지켜보기**

- 배포: 컨테이너화된 애플리케이션을 배포하여 누구나 접근할 수 있게 하는 작업
- 모니터링: 애플리케이션/운영 로그를 분석하여 문제 확인 후 디버깅
- 사용 도구: Render - 최소한의 배포 환경으로 간단함
 - GCP + IaC (Infrastructure as Code) : 복잡함, 수업 X

CI/CD

- 우리가 지금까지 한 것 = CI

- 테스트 코드 작성과 자동화
- GitHub Actions 기반 CI 환경 구축

▶ 코드 변경이 발생할 때 마다, (코드 빌드-)테스트 실행-오류 알람이 자동으로 수행됨

- 그렇다면, 테스트까지 다 통과한 이 코드는 어디서 실행하는가?

- 해당 서비스가 사용자에게 자동으로 제공되는 것은 아님
- 즉, 서버에 배포하는 과정을 개발자가 수행해야 함

▶ 이 과정도 자동화하자! = CD

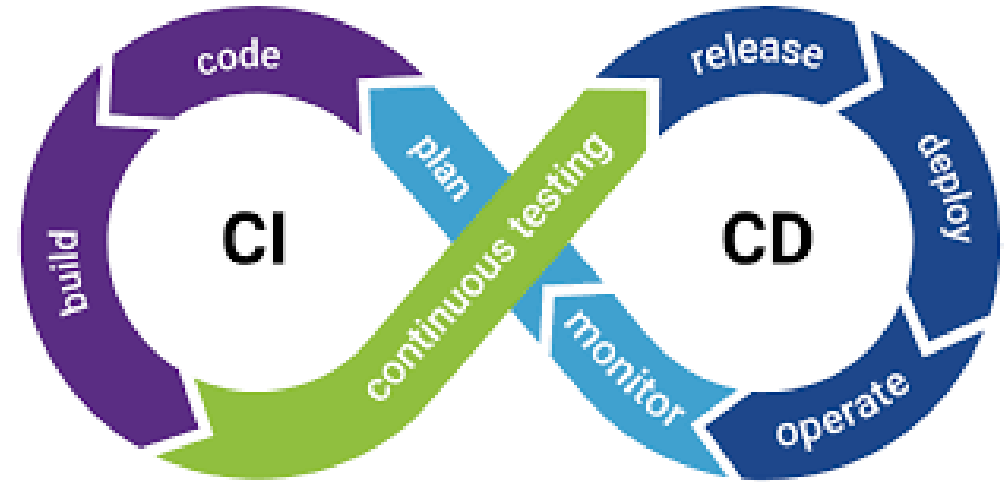
CI/CD

- CI (Continuous Integration, 지속적 통합)

- 코드 변경 사항을 지속적이고 자동으로 통합
 - 자동화된 빌드 + 자동화된 테스트 + 빠른 피드백

- CD (Continuous Delivery/Deployment, 지속적 전달/배포)

- 지속적 전달: 지속적으로 배포가 가능하게 전달
- 지속적 배포: 운영 환경에 지속적으로 배포
 - 예) pypi에 배포, 쿠버네티스에 배포, 클라우드에 배포 등



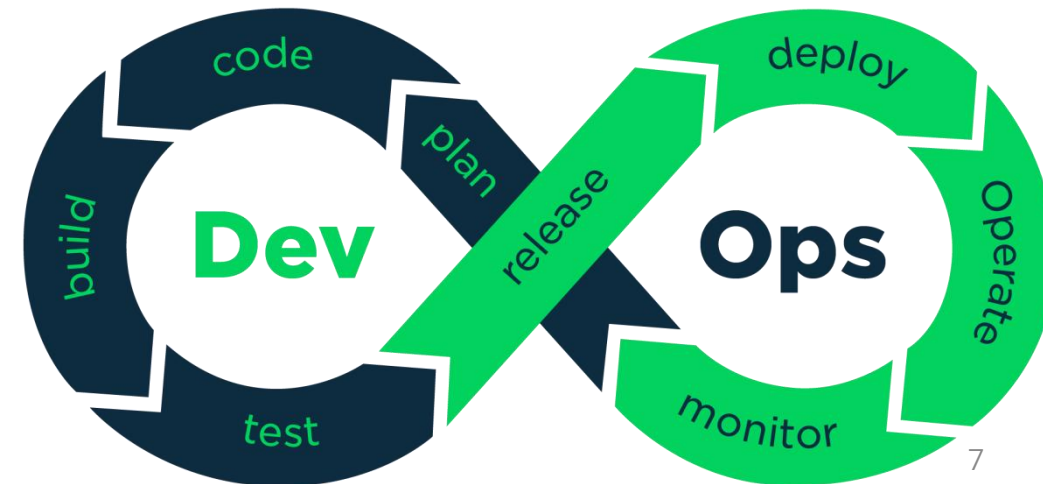
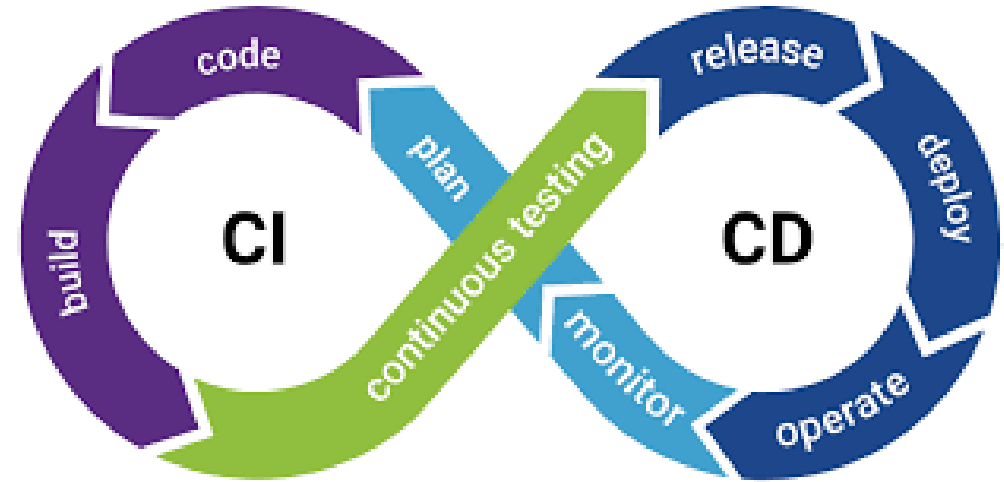
CI/CD와 DevOps

- CI/CD

- 개발/배포 프로세스를 강조
- 코드를 작성하면 빌드-테스트-배포까지 자동화

- DevOps

- CI/CD를 구현하고 조직 전체적인 변화와 협력을 강조

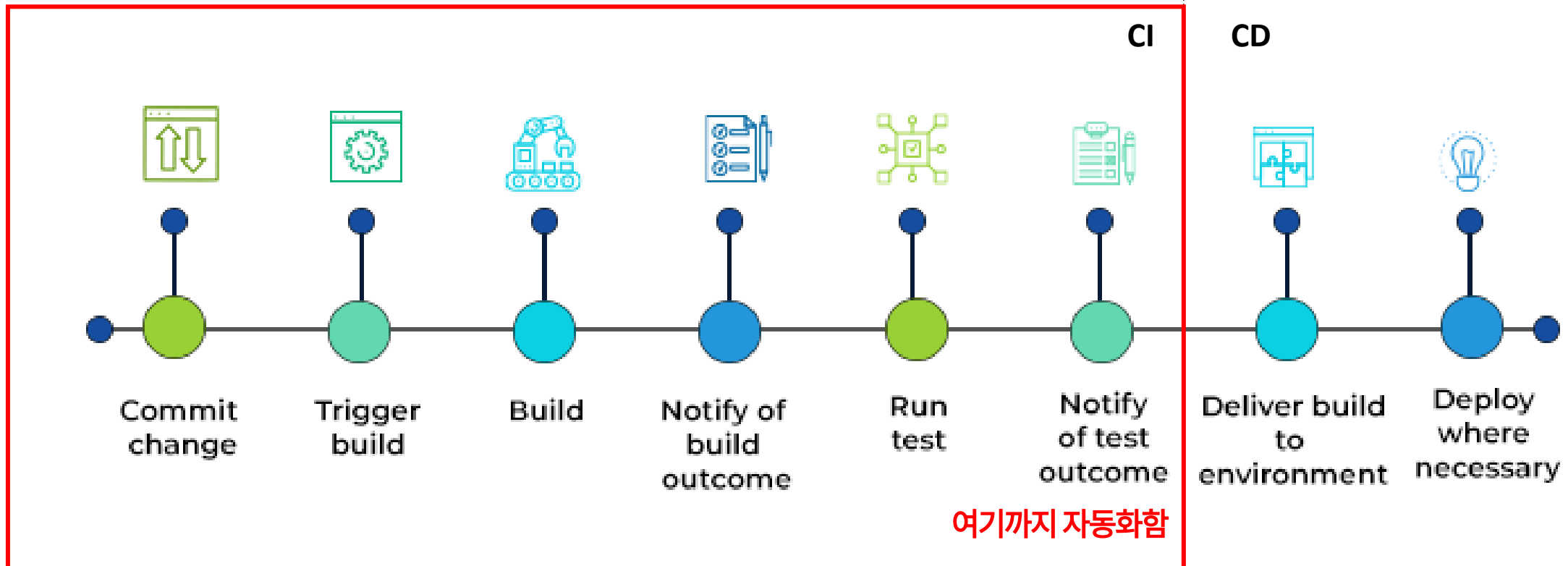


CI/CD 파이프라인

- 파이프라인: 특정 작업의 연속적이고 자동화된 처리 흐름
 - vs 프로세스: 진행되는 절차



CI/CD PIPELINE



CD (지속적 전달/배포)

- SW를 지속적으로

배포하는 작업

- 전달 (delivery):

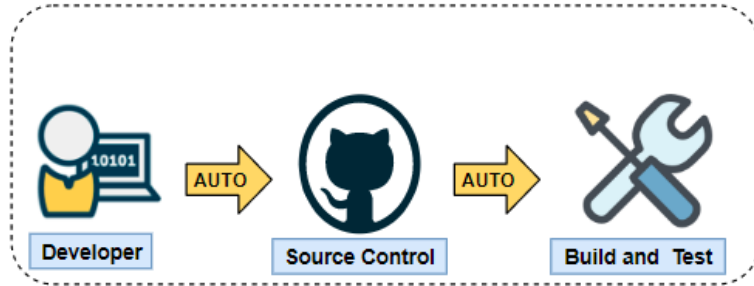
인수 테스트 후 배포 준비

- 인수 테스트: 운영 환경과 유사한 테스트 환경에 배포 후 테스트

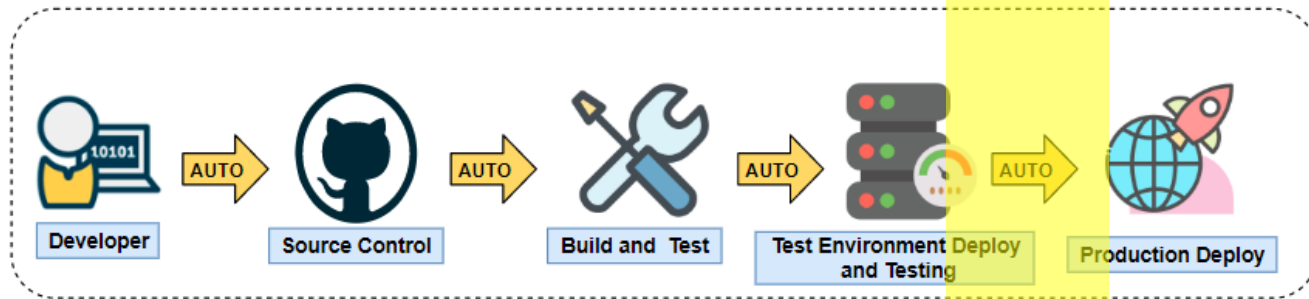
- 배포 (deployment):

실제 운영 환경에 배포

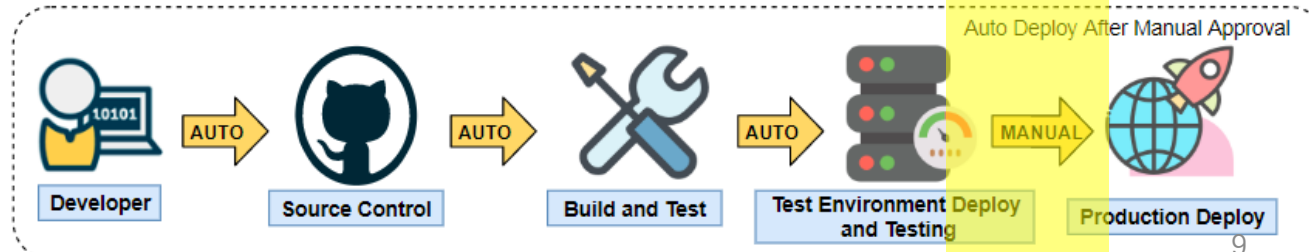
Continuous Integration



Continuous Deployment



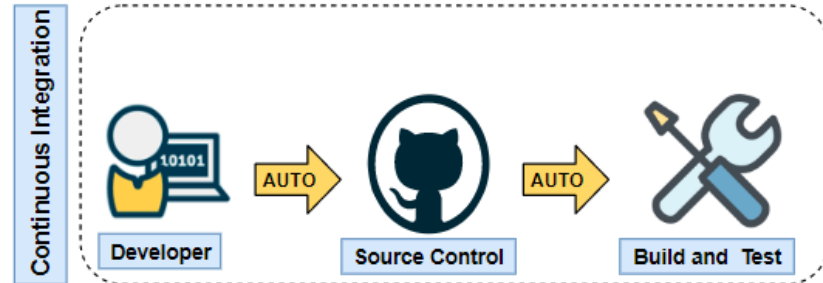
Continuous Delivery



지속적 전달 vs 지속적 배포

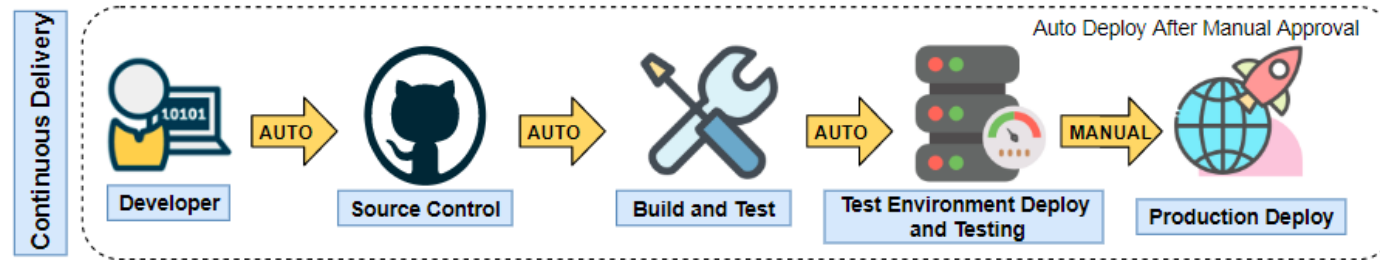
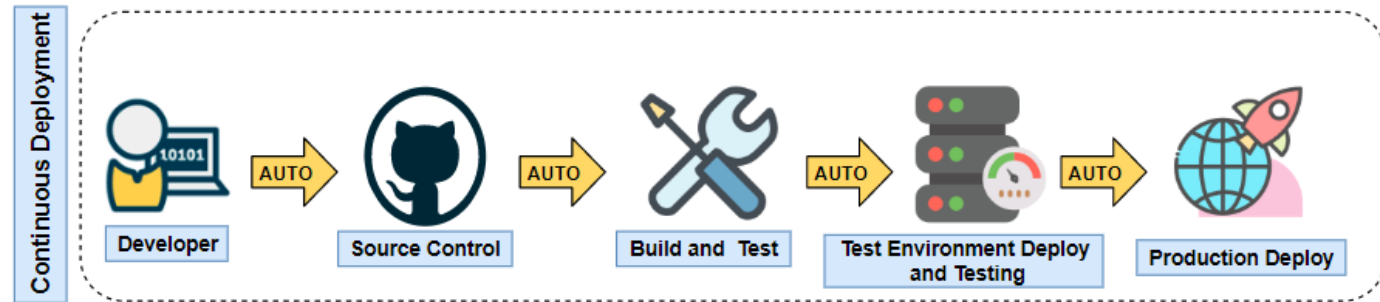
• Continuous Delivery (지속적 전달)

- 코드가 항상 배포 가능한 상태로 유지되도록 함
 - 실제 배포는 사람이 승인해서 수행
- = 배포 준비까지만 자동화!



• Continuous Deployment (지속적 배포)

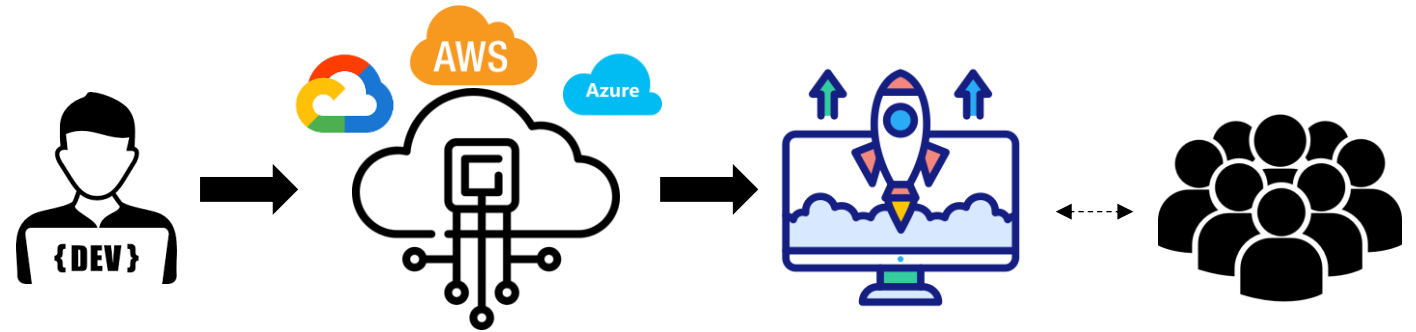
- 테스트를 통과한 코드가 자동으로 배포됨
 - 사람의 개입 없이 서비스에 즉시 반영됨
- = 배포까지 자동화



CD의 필요성

• 전통적인 배포 방식

- 서버에 직접 접속하여 코드 업로드
- 환경 설정을 수동으로 수행
- 사람이 직접 실행-테스트를 수행
- 반복적인 작업으로 시간 낭비
- 사람의 실수로 인한 오류 발생



SW 변경될 때 마다 이 작업 계속..

CD의 필요성

- Knight Capital – 4400억원 손실

- 자동 매매 시스템 업데이트 후 오류 발생 → 일부 서버만 업데이트됨 (배포 불일치)

- 오래된 코드가 일부 서버에서 실행됨

- 핵심 이슈: 테스트 부족, 배포 자동화 부족

- https://en.wikipedia.org/wiki/Knight_Capital_Group



자동 배포 도입의 장점

- 코드 변경 시 자동으로 테스트 후 배포 수행
- 빠른 서비스 반영 가능
- 반복 작업 제거 → 생산성 향상
- 배포 과정의 일관성 유지
- 오류 발생 가능성 감소

표 4-1 주요 IT 서비스 기업의 소프트웨어 배포 현황

회사	배포 주기	배포 소요 시간	신뢰성	사용자 응답
아마존	23,000/day	minutes	high	high
구글	5,500/day	minutes	high	high
넷플릭스	500/day	minutes	high	high
페이스북	1/day	hours	high	high
트위터	3/week	hours	high	high
일반 회사	1/9 months	months/quarters	low/medium	low/medium

우리가 사용할 배포 플랫폼 – Render

- 클라우드 기반의 애플리케이션 배포 플랫폼

- Platform as a Service (PaaS)

- 개발자가 인프라를 관리하지 않고, application 개발에만 집중할 수 있도록 지원하는 클라우드 서비스 모델

- 서버 직접 관리할 필요 없음

- GitHub 저장소와 연동 쉬움

- 코드 변경 시 자동 배포 지원

- 웹 서비스, API 서버 등을 쉽게 배포 가능

= 코드를 올리면 자동으로 서버에서 실행해주는 서비스

실습 8. Render 배포

배포 - 우리는 Render 이용

- 실행 파일을 올리고, 서버에서 계속 실행되게 운영 상태로 만드는 것
- Github에 push → Render가 자동으로 빌드/실행/로그 제공
- 로그를 보고 문제 있으면 고쳐서 다시 push → 재배포
- Render
 - GitHub 저장소를 연결하면, push를 트리거로 빌드/배포를 자동화해주는 PaaS
 - 웹 서비스를 public URL로 띄우고 로그를 확인하고 재배포까지 한번에
- Render + FastAPI
 - Render가 저장소 가져옴 (clone) → requirement.txt로 의존성 설치
→ 우리가 지정한 start command로 서버 실행 → Render가 할당한 포트에 들어오는 요청을 앱으로 전달
 - Uvicorn ... --host 0.0.0.0 -port \$PORT 형태

사전 점검

가상환경

- Pip install -r requirements.txt
- uvicorn app.main:app --reload

```
(class_py) C:\Users\user\OneDrive\바탕 화면\강의\26-1학기\파이프라인(화목4)\실습\2_initial SpamCheck_git>pip install -r requirements.txt
Requirement already satisfied: fastapi in c:\users\user\anaconda3\envs\class_py\lib\site-packages (from -r requirements.txt (line 1)) (0.128.0)
Requirement already satisfied: uvicorn in c:\users\user\anaconda3\envs\class_py\lib\site-packages (from -r requirements.txt (line 2)) (0.40.0)
Requirement already satisfied: python-multipart in c:\users\user\anaconda3\envs\class_py\lib\site-packages (from -r requirements.txt (line 3)) (0.0.21)
Requirement already satisfied: pydantic in c:\users\user\anaconda3\envs\class_py\lib\site-packages (from -r requirements.txt (line 4)) (2.12.5)
Requirement already satisfied: pytest in c:\users\user\anaconda3\envs\class_py\lib\site-packages (from -r requirements.txt (line 5)) (9.0.2)
Requirement already satisfied: httpx in c:\users\user\anaconda3\envs\class_py\lib\site-packages (from -r requirements.txt (line 6)) (0.28.1)
Requirement already satisfied: starlette<0.51.0,>=0.40.0 in c:\users\user\anaconda3\envs\class_py\lib\site-packages (from fastapi->-r requirements.txt (line 1)) (0.50.0)
Requirement already satisfied: typing-extensions>=4.8.0 in c:\users\user\anaconda3\envs\class_py\lib\site-packages (from fastapi->-r requirements.txt (line 1)) (4.15.0)
Requirement already satisfied: annotated-doc>=0.0.2 in c:\users\user\anaconda3\envs\class_py\lib\site-packages (from fastapi->-r requirements.txt (line 1)) (0.0.4)
Requirement already satisfied: anyio<5,>=3.6.2 in c:\users\user\anaconda3\envs\class_py\lib\site-packages (from starlette<0.51.0,>=0.40.0->fastapi->-r requirements.txt (li
Requirement already satisfied: idna>=2.8 in c:\users\user\anaconda3\envs\class_py\lib\site-packages (from anyio<5,>=3.6.2->starlette<0.51.0,>=0.40.0->fastapi->-r requirem
Requirement already satisfied: click>=7.0 in c:\users\user\anaconda3\envs\class_py\lib\site-packages (from uvicorn->-r requirements.txt (line 2)) (8.3.1)
Requirement already satisfied: h11>=0.8 in c:\users\user\anaconda3\envs\class_py\lib\site-packages (from uvicorn->-r requirements.txt (line 2)) (0.16.0)
Requirement already satisfied: annotated-types>=0.6.0 in c:\users\user\anaconda3\envs\class_py\lib\site-packages (from pydantic->-r requirements.txt (line 4)) (0.7.0)
Requirement already satisfied: pydantic-core==2.41.5 in c:\users\user\anaconda3\envs\class_py\lib\site-packages (from pydantic->-r requirements.txt (line 4)) (2.41.5)
Requirement already satisfied: typing-inspection>=0.4.2 in c:\users\user\anaconda3\envs\class_py\lib\site-packages (from pydantic->-r requirements.txt (line 4)) (0.4.2)
Requirement already satisfied: colorama>=0.4 in c:\users\user\anaconda3\envs\class_py\lib\site-packages (from pytest->-r requirements.txt (line 5)) (0.4.6)
Requirement already satisfied: iniconfig>=1.0.1 in c:\users\user\anaconda3\envs\class_py\lib\site-packages (from pytest->-r requirements.txt (line 5)) (2.3.0)
Requirement already satisfied: packaging>=22 in c:\users\user\anaconda3\envs\class_py\lib\site-packages (from pytest->-r requirements.txt (line 5)) (25.0)
Requirement already satisfied: pluggy<2,>=1.5 in c:\users\user\anaconda3\envs\class_py\lib\site-packages (from pytest->-r requirements.txt (line 5)) (1.6.0)
Requirement already satisfied: pygments>=2.7.2 in c:\users\user\anaconda3\envs\class_py\lib\site-packages (from pytest->-r requirements.txt (line 5)) (2.19.1)
Requirement already satisfied: certifi in c:\users\user\anaconda3\envs\class_py\lib\site-packages (from httpx->-r requirements.txt (line 6)) (2026.1.4)
Requirement already satisfied: httpcore==1.* in c:\users\user\anaconda3\envs\class_py\lib\site-packages (from httpx->-r requirements.txt (line 6)) (1.0.9)
```

```
(class_py) C:\Users\user\OneDrive\바탕 화면\강의\26-1학기\파이프라인(화목4)\
INFO: Will watch for changes in these directories: ['C:\\Users\\user\\One
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [9576] using StatReload
INFO: Started server process [676]
INFO: Waiting for application startup.
INFO: Application startup complete.
^
```

127.0.0.1:8000/docs

127.0.0.1:8000



스팸 판별기

안녕!

스팸 탐지

결과: ham (score=0)

127.0.0.1:8000



스팸 판별기

Get a free prize!

스팸 탐지

결과: spam (score=2)



SpamCheck Web 0.1.0 OAS 3.1

/openapi.json

default

GET / Home

POST /classify Classify

Schemas

ClassifyRequest > Expand all object

HTTPValidationError > Expand all object

ValidationError > Expand all object

Render (https://render.com/)

- 가입 후 github 계정 가입 → web service 선택

The screenshot displays the Render website's user interface. At the top, a purple banner promotes migrating apps from Heroku. Below this, the navigation bar includes the Render logo, a menu icon, and links for 'Contact', 'Sign In', and 'Get Started'. The main content area shows the 'Create an account' page, which is partially obscured by a GitHub authorization overlay. This overlay, titled 'Render by Render would like permission to:', lists permissions such as verifying GitHub identity, accessing resources, and acting on behalf of the user. It also shows resources on the account, including email addresses. To the right of the overlay, the 'Create a new Service' page is visible, featuring a progress bar with steps: '1 Choose service', '2 Configure', and '3 Deploy'. Two service options are presented: 'Static Sites' (described as static content served over a global CDN) and 'Web Services' (described as dynamic web apps). Both options include links to create new instances. The bottom of the image features the Chonnam National University logo and name in Korean and English.

Migrate apps from Heroku with near-zero downtime. [Heroku Migration Docs >](#)

Render Contact Sign In **Get Started**

Render

Create an account

GitHub

Render by **Render** would like permission to:

- Verify your GitHub identity (MisookKim)
- Know which resources you can access
- Act on your behalf [Learn more](#)

Resources on your account

- Email addresses (read)
View your email addresses

[Learn more about Render](#)

Cancel Authorize Render

Create a new Service [Skip](#)

1 Choose service > 2 Configure > 3 Deploy [Which to use?](#)

Static Sites
Static content served over a global CDN. Ideal for frontend, blogs, and content sites.
[New Static Site →](#)

Web Services
Dynamic web app. Ideal for full-stack apps, API servers, and mobile backends.
[New Web Service →](#)

CHONNAM NATIONAL UNIVERSITY

Render (https://render.com/)

• GitHub 연동

The collage illustrates the steps to connect a GitHub account to Render:

- No repositories found:** A screen prompting the user to connect a Git provider (GitLab or Bitbucket) to deploy from existing repositories.
- Select user to authorize:** A screen showing the user is signed in as 'MisooKim' and offers to select another user ('MisooKim2').
- Install on your personal account:** A screen for installing the integration on the personal account of 'Misoo (Rose) Kim'. It allows selecting 'All repositories' or 'Only select repositories'. A dropdown menu shows a list of repositories like 'MisooKim/AdaptationEngine', 'MisooKim/AdaptiveRobot', etc.
- Selected 1 repository:** A screen showing the selected repository 'MisooKim/jnu2603-spamchecker_test' and the permissions granted to Render, such as 'Read access to Dependabot alerts, code, and metadata' and 'Read and write access to actions, checks, commit statuses, deployments, environments, issues, pull requests, repository hooks, and workflows'.

이메일 인증..

Render (http)

• Web service 생성

Name
A unique name for your web service.

jnu2603-spamchecker_test

Language
Choose the [runtime environment](#) for this service.

Python 3

Branch
The Git branch to build and deploy.

main

Region
Your services in the same [region](#) can communicate over a [private network](#).

Oregon (US West)

Root Directory Optional
If set, Render runs commands from this directory instead of the repository root. Most commonly used with a [monorepo](#).

e.g. src

Build Command
Render runs this command to build your app before each deploy.

\$ pip install -r requirements.txt

Start Command
Render runs this command to start your app with each deploy.

\$ uvicorn app.main:app --host 0.0.0.0 --port \$PORT

My Workspace New Web Service Search + New Upgrade

New Web Service

Source Code [Git Provider](#) [Public Git Repository](#) [Existing Image](#)

Search Credentials (1)

MisookKim / jnu2603-spamchecker_test 14h ago

For hobby projects

Free 512 MB (RAM)
\$0 / month 0.1 CPU

For professional use
For more power and to get the most out of Render, we recommend using one of our paid instance types. All paid instances support:

- Zero Downtime
- SSH Access
- Scaling
- One-off jobs
- Support for persistent disks

Starter 512 MB (RAM)
\$7 / month 0.5 CPU

Pro 4 GB (RAM)
\$85 / month 2 CPU

Pro Max 16 GB (RAM)
\$225 / month 4 CPU

[Need a custom instance type?](#) We support...

Environment Variables
Set environment-specific config and secrets (such as API keys), then read those values from your code. [Learn more](#).

NAME_OF_VARIABLE value

+ Add Environment Variable Add from .env

Advanced

Deploy Web Service

All logs Search Live tail

Feb 11

12:43:47 PM Downloading packaging-26.0-py3-none-any.whl (74 kB)

12:43:47 PM Downloading pygments-2.19.2-py3-none-any.whl (1.2 MB)

12:43:47 PM 13.3 MB/s eta 0:00:00 1.2/1.2 MB

12:43:47 PM Downloading typing_extensions-4.15.0-py3-none-any.whl (44 kB)

12:43:47 PM Downloading typing_inspection-0.4.2-py3-none-any.whl (14 kB)

12:43:47 PM Downloading certifi-2026.1.4-py3-none-any.whl (152 kB)

12:43:47 PM Installing collected packages: typing-extensions, python-multipart, pygments, pluggy, packaging, iniconfig, idna, h11, click, certifi, annotated-types, annotated-doc, uvicorn, typing-inspection, pytest, pydantic-core, httpcore, anyio, starlette, pydantic, httpx, fastapi

12:43:51 PM Successfully installed annotated-doc-0.0.4 annotated-types-0.7.0 anyio-4.12.1 certifi-2026.1.4 click-8.3.1 fastapi-0.128.7 h11-0.16.0 httpcore-1.0.9 httpx-0.28.1 idna-3.11 iniconfig-2.3.0 packaging-26.0 pluggy-1.6.0 pydantic-2.12.5 pydantic-core-2.41.5 pygments-2.19.2 pytest-9.0.2 python-multipart-0.0.22 starlette-0.52.1 typing-extensions-4.15.0 typing-inspection-0.4.2 uvicorn-0.40.0

12:43:52 PM [notice] A new release of pip is available: 25.1.1 -> 26.0.1

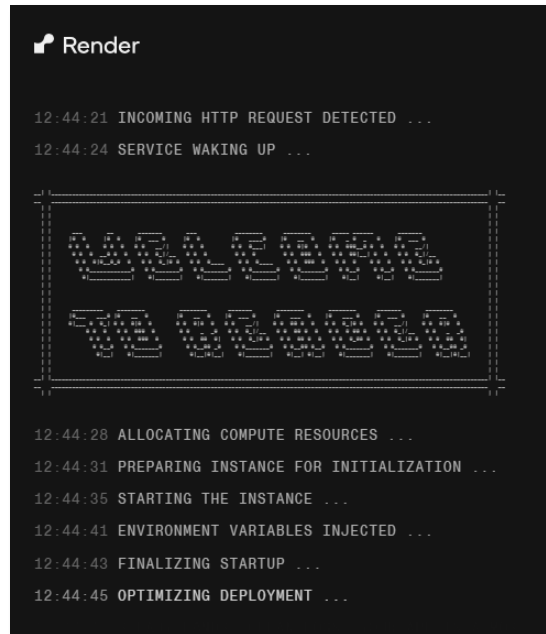
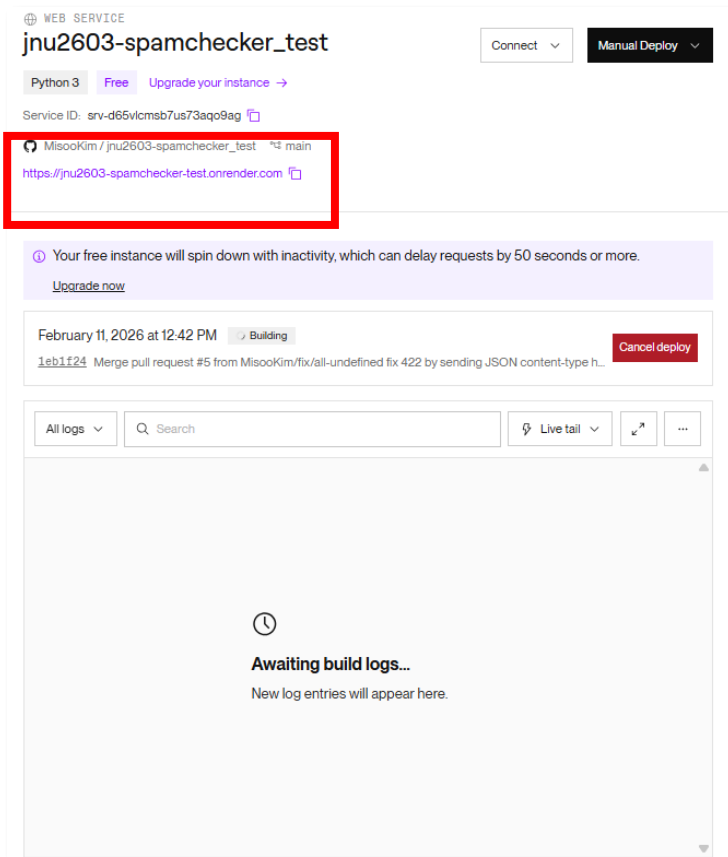
12:43:52 PM [notice] To update, run: pip install --upgrade pip

12:44:02 PM ==> Uploading build...

uvicorn app.main:app --host 0.0.0.0 --port \$PORT

Render (https://render.com/)

- 서비스 URL 확인 후 실행



Render (https://render.com/)

My Workspace

My project / Production / jnu2603-spamchecker_test

Environment: jnu2603-spamchecker_test

Events

Settings

MONITOR

Logs

Changelog

Invite a friend

Contact support

Render Status

Need better ways to work with logs? Try the [Render CLI](#), [Render MCP Server](#), or set up a [log stream integration](#)

```
12:45:14 PM [2k16g] INFO: 168.131.150.55:0 - "HEAD / HTTP/1.1" 405 Method Not Allowed
12:45:14 PM ==>
12:45:14 PM ==> =====
12:45:14 PM [2k16g] INFO: 35.197.97.226:0 - "GET / HTTP/1.1" 200 OK
12:45:14 PM ==>
12:45:14 PM ==> Available at your primary URL https://jnu2603-spamchecker-test.onrender.com
12:45:14 PM ==>
12:45:14 PM ==> =====
12:45:14 PM [2k16g] INFO: 168.131.150.55:0 - "GET / HTTP/1.1" 200 OK
12:45:51 PM [2k16g] INFO: 168.131.150.55:0 - "GET / HTTP/1.1" 200 OK
12:46:39 PM [2k16g] INFO: 168.131.150.55:0 - "POST /classify HTTP/1.1" 200 OK
12:46:47 PM [2k16g] INFO: 168.131.150.55:0 - "POST /classify HTTP/1.1" 200 OK
12:46:53 PM [2k16g] INFO: 168.131.150.55:0 - "GET / HTTP/1.1" 200 OK
12:47:03 PM [2k16g] INFO: 168.131.150.55:0 - "POST /classify HTTP/1.1" 200 OK
12:47:10 PM [2k16g] INFO: 168.131.150.55:0 - "POST /classify HTTP/1.1" 200 OK
```

스팸 판별기

Get a free prize!

스팸 탐지

결과: spam (score=2)

스팸 판별기

안녕!

스팸 탐지

결과: ham (score=0)

```
==> =====
[2k16g] INFO: 168.131.150.55:0 - "GET / HTTP/1.1" 200 OK
[2k16g] INFO: 168.131.150.55:0 - "GET / HTTP/1.1" 200 OK
[2k16g] INFO: 168.131.150.55:0 - "POST /classify HTTP/1.1" 200 OK
[2k16g] INFO: 168.131.150.55:0 - "POST /classify HTTP/1.1" 200 OK
[2k16g] INFO: 168.131.150.55:0 - "GET / HTTP/1.1" 200 OK
[2k16g] INFO: 168.131.150.55:0 - "POST /classify HTTP/1.1" 200 OK
[2k16g] INFO: 168.131.150.55:0 - "POST /classify HTTP/1.1" 200 OK
[2k16g] INFO: 115.145.178.157:0 - "GET / HTTP/1.1" 200 OK
[2k16g] INFO: 115.145.178.157:0 - "GET /favicon.ico HTTP/1.1" 404 Not Found
[2k16g] INFO: 115.145.178.157:0 - "POST /classify HTTP/1.1" 200 OK
[2k16g] INFO: 168.131.150.55:0 - "POST /classify HTTP/1.1" 200 OK
```

ways to work with logs? Try the [Render CLI](#), [Render MCP Server](#), or set up a [log stream integration](#)

스팸 판별기

안녕!

스팸 탐지

결과: ham (score=0)

스팸 판별기

메롱!

스팸 탐지

결과: ham (score=0)

Render (<https://render.com/>)

스팸 판별기

- **운영 중 에러 발생!** : 뜯어쓰기만 넣고 탐지하면?

```
12:50:01 PM [2k10g] INFO: 168.131.150.55:0 - "POST /classify HTTP/1.1" 500 Internal Server Error
12:50:01 PM [2k10g] ERROR: Exception in ASGI application
12:50:01 PM [2k10g] Traceback (most recent call last):
12:50:01 PM [2k10g]   File "/opt/render/project/src/.venv/lib/python3.13/site-packages/uvicorn/protocols/http/h11_impl.py", line 410, in run_asgi
12:50:01 PM [2k10g]       result = await app(  # type: ignore[func-returns-value]
12:50:01 PM [2k10g]           ~~~~~
12:50:01 PM [2k10g]       self.scope, self.receive, self.send
12:50:01 PM [2k10g]       ~~~~~
12:50:01 PM [2k10g]   )
12:50:01 PM [2k10g]   ^
12:50:01 PM [2k10g]   File "/opt/render/project/src/.venv/lib/python3.13/site-packages/uvicorn/middleware/proxy_headers.py", line 60, in __call__
12:50:01 PM [2k10g]       return await self.app(scope, receive, send)
12:50:01 PM [2k10g]           ~~~~~
12:50:01 PM [2k10g]   File "/opt/render/project/src/.venv/lib/python3.13/site-packages/fastapi/applications.py", line 1138, in __call__
```

```
12:50:01 PM [2kl6g] File "/opt/render/project/src/.venv/lib/python3.13/site-
packages/starlette_exception_handler.py", line 42, in wrapped_app
12:50:01 PM [2kl6g]     await app(scope, receive, sender)
12:50:01 PM [2kl6g] File "/opt/render/project/src/.venv/lib/python3.13/site-
packages/fastapi/routing.py", line 107, in app
12:50:01 PM [2kl6g]     response = await f(request)
12:50:01 PM [2kl6g]     ^^^^^^^^^^^^^^^^^^^^^
12:50:01 PM [2kl6g] File "/opt/render/project/src/.venv/lib/python3.13/site-
packages/fastapi/routing.py", line 426, in app
12:50:01 PM [2kl6g]     raw_response = await run_endpoint_function(
12:50:01 PM [2kl6g]     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
12:50:01 PM [2kl6g]     ...<3 lines>...
12:50:01 PM [2kl6g] )
12:50:01 PM [2kl6g] ^
12:50:01 PM [2kl6g] File "/opt/render/project/src/.venv/lib/python3.13/site-
packages/fastapi/routing.py", line 314, in run_endpoint_function
12:50:01 PM [2kl6g]     return await dependant.call(**values)
12:50:01 PM [2kl6g]     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
12:50:01 PM [2kl6g] File "/opt/render/project/src/app/main.py", line 25, in classify
```

스팸 탐지

에러 발생

```
12:50:01 PM [2kl6g] raw_response = await run_endpoint_function(
12:50:01 PM [2kl6g]
12:50:01 PM [2kl6g] ...<3 lines>...
12:50:01 PM [2kl6g] )
12:50:01 PM [2kl6g] ^
12:50:01 PM [2kl6g] File "/opt/render/project/src/.venv/lib/python3.13/site-
12:50:01 PM [2kl6g] packages/fastapi/routing.py", line 314, in run_endpoint_function
12:50:01 PM [2kl6g]     return await dependant.call(**values)
12:50:01 PM [2kl6g]
12:50:01 PM [2kl6g] File "/opt/render/project/src/app/main.py", line 25, in classify
12:50:01 PM [2kl6g]     label, score = check_spam(text)
12:50:01 PM [2kl6g]
12:50:01 PM [2kl6g] ValueError: too many values to unpack (expected 2)
12:50:10 PM ==> Detected service running on port 10000
12:50:10 PM ==> Docs on specifying a port: https://render.com/docs/web-services#port-binding
```


Render (<https://render.com/>)

- 에러 원인: dict 반환 안함

```
2_initial SpamCheck_git > app > spam.py > check_spam.py
1 # app/spam.py
2 def check_spam(text: str) -> str:
3     text = text.lower().strip()
4     if text == "":
5         return "ham"
6
```

- 수정 후 다시 push 해보자

```
4     if text == "":
5         return "ham", 0
```

- 수정 → `git add .` → `git commit -m "fix 에러발생"` → `git push origin main`

WEB SERVICE

jnu2603-spamchecker_test

Python 3

Free

Upgrade your instance →

Service ID: `srv-d65vlcmsb7us73aqo9ag`

MisookKim / jnu2603-spamchecker_test

<https://jnu2603-spamchecker-test.onrender.com>

ⓘ Your free instance will spin down with inactivity, which can delay requests by 50 seconds or more.

[Upgrade now](#)

Filter events

31

Deploy started for `39d4f28f`: fix 에러발생

New commit via Auto-Deploy

February 11, 2026 at 12:56 PM

Cancel deploy

```
12:57:39 PM      >>> Build successful 🎉
12:58:05 PM      >>> Deploying...
12:58:05 PM      >>> Setting WEB_CONCURRENCY=1 by default, based on available CPUs in
instance
12:58:30 PM [98b84c] >>> Running 'uvicorn app.main_app --host 0.0.0.0 --port 8080'
12:58:35 PM [98b84c] INFO: Started server process [557]
12:58:35 PM [98b84c] INFO: Waiting for application startup.
12:58:35 PM [98b84c] INFO: Application startup complete.
12:58:35 PM [98b84c] INFO: Uvicorn running on http://0.0.0.0:8080 (Press CTRL+C to quit)
12:58:35 PM [98b84c] INFO: 127.0.0.1:38776 - "HEAD / HTTP/1.1" 405 Method Not Allowed
12:58:36 PM      >>> Your service is live 🎉
12:58:36 PM [98b84c] INFO: 35.247.111.159:0 - "GET / HTTP/1.1" 200 OK
12:58:37 PM      >>>
12:59:37 PM      >>> #####
```

스팸 판별기

스팸 탐지

결과: ham (score=0)

숙제 없음

- 그러나 오늘 한 것에 이어서 목요일에 할 것이니 꼭 해놓기.

도커 미리 사용해보기

가상환경 - 도커 사용해보기 (윈도우)

1. 도커 설치: <https://www.docker.com/>

- wsl 추가 설치됨

2. docker run hello-world

1. 다운로드 받은 이미지 중 hello-world가 있는지 확인, 있으면 바로 실행
2. 없으면 도커 허브에서 hello-world 다운로드 받고 실행

```
C:\Users\user>docker run hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/

C:\Users\user>
```

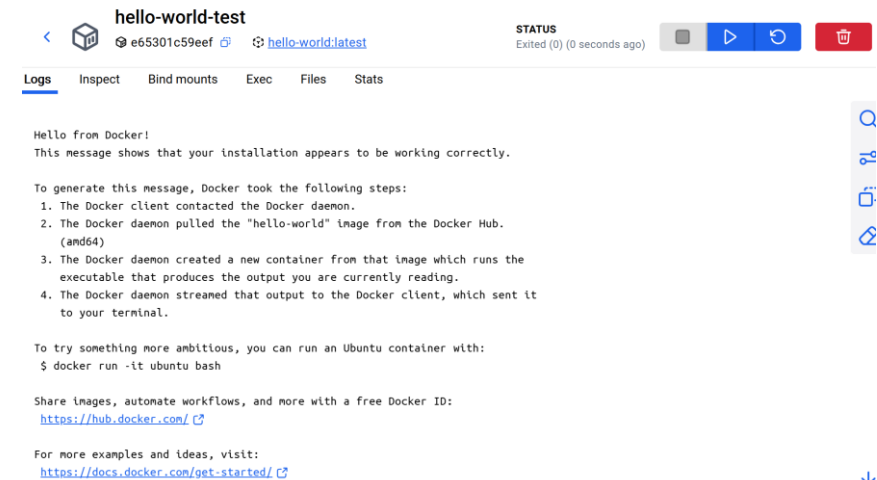
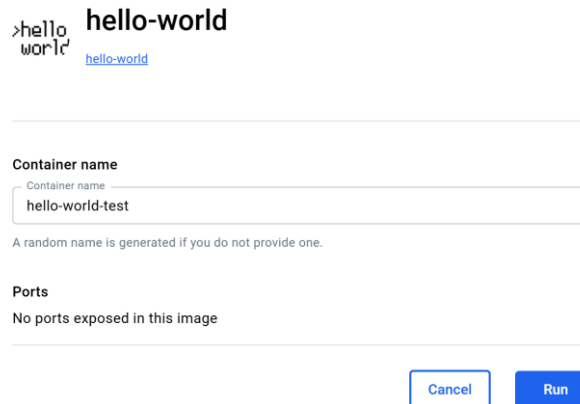
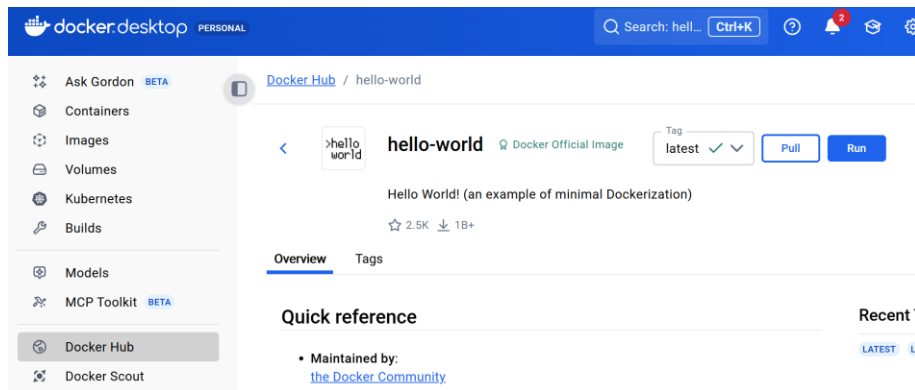
가상환경 - 도커 사용해보기 (윈도우)

1. 도커 설치: <https://www.docker.com/>

- wsl 추가 설치됨

2. 도커 데스크탑에서 hello-world 실행해보기

- Hello-world 검색 후 실행 시도 → 컨테이너 생성 → 실행 후 로그 확인



가상환경 - 도커로 flask 웹앱 만들기

• app.py

```
# 파이썬 웹 앱 개발 라이브러리
from flask import Flask

# Flask 애플리케이션 객체 생성
app = Flask(__name__)

# 라우팅(Route) 설정
# 사용자가 웹 브라우저에서 "/" (루트 경로)로 접속하면
# (http://localhost:5000)
# 아래의 hello() 함수가 실행됨
@app.route("/")
def hello():
    return "Hello, Docker! 🚀"

# if __name__ == "__main__": -> 이 파일이 직접 실행될 때만 실행됨
if __name__ == "__main__":
    # Flask 서버 실행
    # host="0.0.0.0" : 모든 네트워크 인터페이스에서 접속 허용
    # port=5000 : 5000번 포트에서 서비스 시작
    app.run(host="0.0.0.0", port=5000)
```

• Dockerfile (UTF-8 저장 필수)

```
# -----
# 1. 베이스 이미지 선택
# - 파이썬 3.9 버전이 설치된 리눅스 환경 사용
# - slim 이미지는 불필요한 요소를 제거해 가볍게 만든 버전
# -----
FROM python:3.9-slim

# -----
# 2. 작업 디렉토리 설정
# - 컨테이너 내부에서 /app 폴더를 기본 작업 위치로 지정
# - 이후 COPY, RUN, CMD 명령어는 모두 /app 기준으로 실행됨
# -----
WORKDIR /app

# -----
# 3. 코드 복사
# - 현재 폴더(호스트)의 app.py 파일을 컨테이너 내부 /app 폴더로 복사
# - 즉, 내가 작성한 프로그램을 이미지 안에 넣는 과정
# -----
COPY app.py .

# -----
# 4. 필요한 라이브러리 설치
# - Flask 웹 프레임워크 설치
# - RUN은 이미지 빌드 과정에서 한 번 실행됨
# -----
RUN pip install flask

# -----
# 5. 컨테이너 시작 시 실행할 명령어
# - CMD는 컨테이너가 실행될 때 수행할 기본 명령어 지정
# - 여기서는 "python app.py"를 실행 → Flask 웹 서버가 시작됨
# -----
CMD ["python", "app.py"]
```

가상환경 - 도커로 flask 웹앱 만들기

- `docker build -t {이미지이름} {컨텍스트}`

- `-t {이름}` : 이미지 이름 설정
- `.` : 현재 디렉토리를 빌드 컨텍스트로 설정

```
C:\Users\user\Desktop\docker_sample>docker build -t myflaskapp .
[+] Building 7.6s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 131B
=> [internal] load metadata for docker.io/library/python:3.9-slim
=> [auth] library/python:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/4] FROM docker.io/library/python:3.9-slim@sha256:cf07045079721
=> => resolve docker.io/library/python:3.9-slim@sha256:cf07045079721
=> [internal] load build context
=> => transferring context: 211B
=> CACHED [2/4] WORKDIR /app
=> [3/4] COPY app.py .
=> [4/4] RUN pip install flask
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:abbd45697ad9cd0ed8037ea19ff74f14a1f
=> => exporting config sha256:72298bc554eb15050b6b08c0d62bacd736cf0
=> => exporting attestation manifest sha256:6744f9eb5f8013cbe4ffb1d
=> => exporting manifest list sha256:dcaab6b628b6d64c4abd385e497be8
=> => naming to docker.io/library/myflaskapp:latest
=> => unpacking to docker.io/library/myflaskapp:latest
```

```
C:\Users\user\Desktop\docker_sample>
```

- `docker run -d -p 5000:5000 --name flask-container myflaskapp`

- `docker run` : 이미지를 실행해 컨테이너 생성
- `-d` : Detached mode (백그라운드 실행)
- `-p 5000:5000` : 포트 매핑 – 내PC포트:컨테이너포트
 - 즉, localhost:5000 에서 컨테이너 안의 flask 서버에 접속됨
- `--name {컨테이너이름}` {실행이미지이름}

```
C:\Users\user\Desktop\docker_sample>docker run -d -p 5000:5000 --name flask-container myflaskapp
393c994fee8429a2368b2ffe968ff498dadd67850334b9699ad3e71a22071f32
C:\Users\user\Desktop\docker_sample>
```

Containers [Give feedback](#)

Container CPU usage ⓘ

0.03% / 2200% (22 CPUs available)

Container memory usage ⓘ

20.98MB / 30.37GB

[Show charts](#)

Search



Only show running containers

☐	Name	Container ID	Image	Port(s)	Actions
☐	flask-container	393c994fee84	myflaskapp	5000:5000	

가상환경 - 도커로 flask 웹앱 만들기

[Containers](#) / flask-container

flask-container

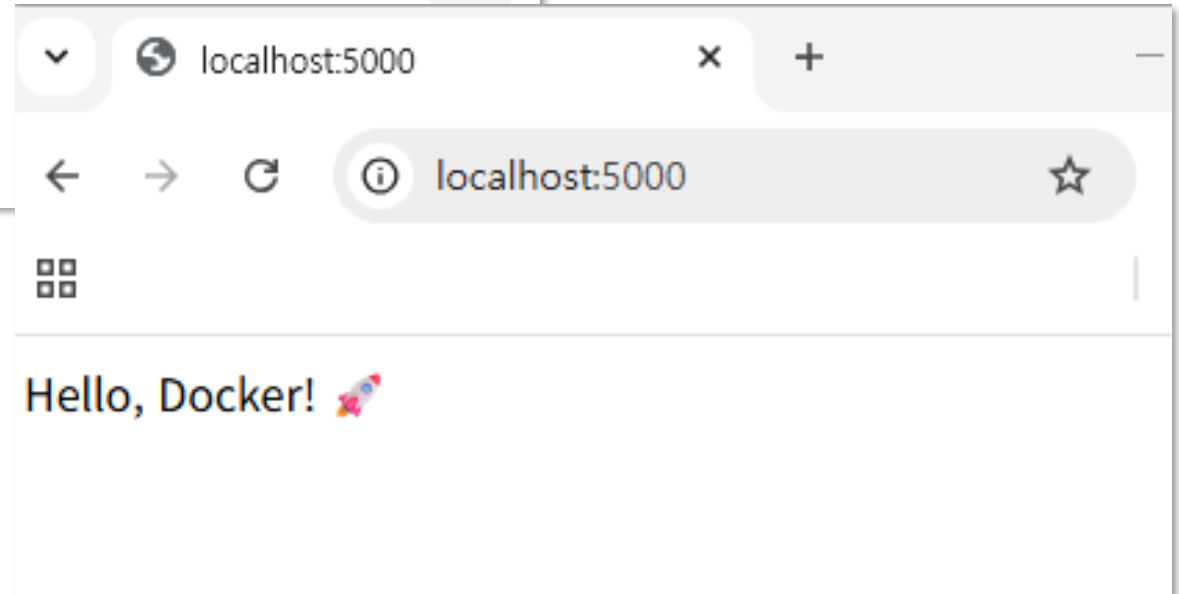
<  393c994fee84  [myflaskapp](#) STATUS
Running (33 seconds ago)    

[5000:5000](#) 

Logs Inspect Bind mounts Exec Files Stats

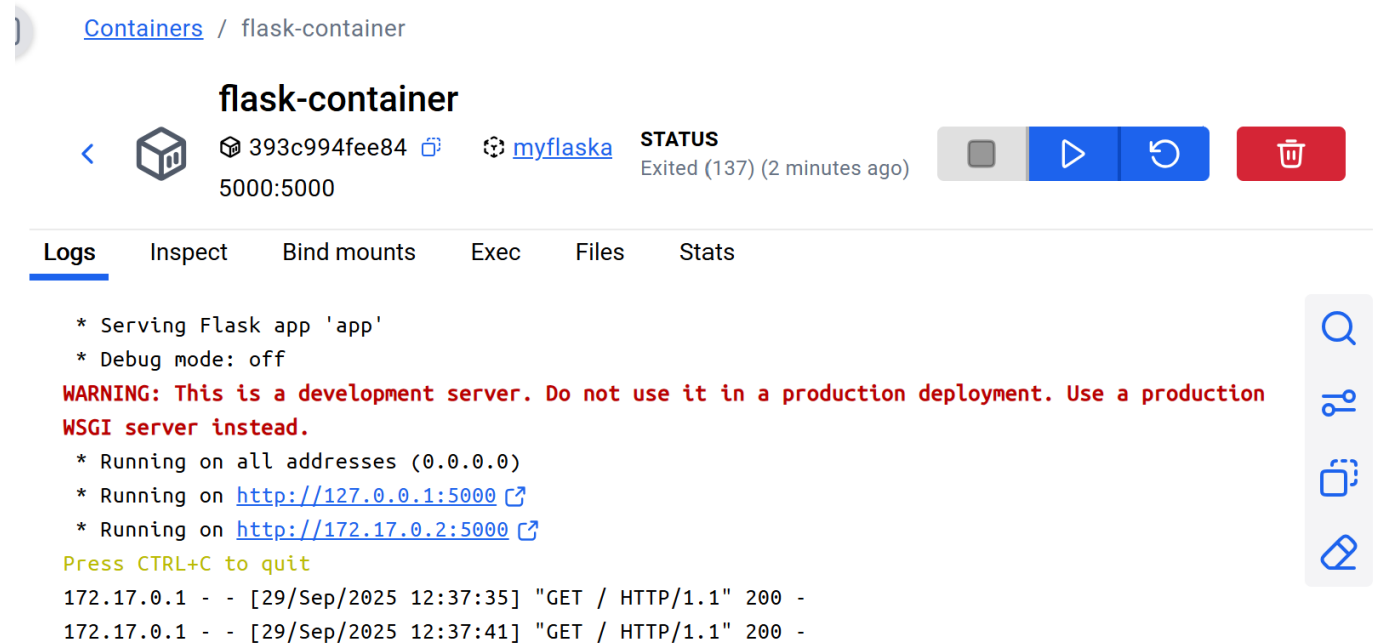
- * Serving Flask app 'app'
- * Debug mode: off
- WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.**
- * Running on all addresses (0.0.0.0)
- * Running on <http://127.0.0.1:5000> 
- * Running on <http://172.17.0.2:5000> 

Press CTRL+C to quit



가상환경 - 도커

- 도커 데스크탑에서 관리



- 윈도우에서 컨테이너가 저장되는 곳

